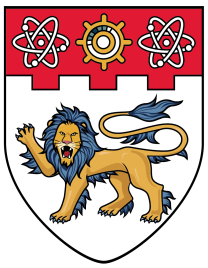

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Infrastructure Provisioning for SpeechLab Applications with Terraform

Chan Ming Han

U2120477J

Supervisor: Prof Chng Eng Siong

A Final Year Report submitted to School of Computer Science and Engineering, Nanyang Technological University in partial fulfilment of the requirements for the Degree of

BACHELOR OF ENGINEERING SCIENCE (COMPUTER ENGINEERING)

2024/2025 Semester 2

Abstract

As companies move toward containerisation technologies to improve scalability and portability in their applications, Kubernetes has emerged as the industry standard for orchestrating containerised workloads. Despite its advantages, deploying containerised workloads to the cloud is highly complex, and resource intensive. Developers and engineer also need to master the various cloud technologies, such as security and networking requirements, often diverting focus from core application development. This project introduces cloudkube, a lightweight, Python-based command-line tool designed to automate the deployment of Kubernetes clusters in the cloud. Inspired by minikube and leveraging Infrastructure as Code (IaC) principles with Terraform, cloudkube abstracts the complexities of cloud provisioning and Kubernetes setup. It enables developers to focus on core application development, rather than cloud or deployment configurations.. The tool supports customizable configurations, automated local environment setup, and idempotent resource provisioning while ensuring low-cost prototyping by leveraging AWS Free Tier resources. Through cloudkube, deployment times are significantly reduced, and infrastructure provisioning is made accessible to users without deep expertise in cloud technologies. This paper details the tool's architecture, implementation, and workflow, showcasing its potential to enhance productivity, reduce operational overhead, and accelerate the deployment of containerised applications to the cloud.

Keywords: Containerization, Cloud Automation, Infrastructure as Code (IaC), Terraform, Cloud Deployment, Kubernetes Orchestration, Developer Productivity, Lightweight CLI Tools, Cloud-Native Applications

Acknowledgement

I would like to express my deepest gratitude to my academic advisors and mentors, including but not limited to Prof Chng Eng Siong, Research Assistant Vu Thy Ly, and James. Their guidance and invaluable insights have been instrumental throughout the course of this project. Their continuous support and constructive feedback have greatly enhanced the quality of my work. I am particularly grateful for their encouragement and inspiration, which motivated me to tackle complex challenges and explore innovative solutions.

I extend my sincere thanks to my peers, who contributed through stimulating discussions and innovative ideas. Their willingness to share knowledge and exchange ideas fostered an enriching environment that enabled the successful completion of this project.

Finally, I would like to acknowledge the funding and infrastructure support provided by Nanyang Technological University. The access to the AWS cloud platform significantly facilitated the implementation and testing phases of this project. Without their generous support, this work would not have been possible.

Table of Contents

Lists of Figures

List of Tables

Introduction

1.1 Background

Automatic Speech Recognition (ASR) is a transformative technology that enables machines to interpret and process human speech into text. Rooted in advancements in machine learning, signal processing, and linguistics, ASR has rapidly evolved over the past decades to become a cornerstone of human-computer interaction. At its core, ASR has allowed for machines to understand, and reproduce human speech artificially [17].

The use cases for ASR span a wide range of real-time applications that significantly impact various industries. In customer service, ASR powers virtual assistants and chatbots, enabling real time transcription of customer queries and automated responses. Virtual assistants like Apple's Siri, Amazon's Alexa, Microsoft's Cortana, and more also rely on ASR to process voice commands and execute tasks ranging from setting reminders to controlling smart home devices [12]. On our favourite video conferencing platforms such as Microsoft Teams, Zoom, or Google Meets, ASR is also integrated to provide live captions during meetings, enhancing collaboration and inclusivity [19]. These examples highlight how ASR has become an integral part of our daily lives, simplifying tasks, improving accessibility, and transforming how people interact with technology.

NTU SpeechLab has also undertaken multiple projects, creating speech engines that can transcribe English, Mandarin and Singlish in real-time. These speech engines are used in various use cases including speech-to-text at call centres, live transcription during interviews, as well as medical consultations. This is done through either of two methods based on the use. case. The first is batch transcription, in which a completed audio recording is analysed for transcription. The second is live transcription, in which an audio stream is analysed in real-time to provide on-the-fly transcription.

Due to the on-the-fly nature of live transcriptions, as well as the ever evolving need for ASR technologies, it is important for such applications to be built in a robust and scalable manner. These services should be highly available and scalable. Previous FYPs have also worked on containerisation and deployment of the ASR speech model via Kubernetes or Docker Swarm, as well as robust CI/CD pipelines using ArgoCD to speed up the build and deployment times the ASR speech application.

However, as the number of such applications have increased, the infrastructure requirements have also become more demanding. In the deployment of the ASR speech model, one of the greatest pain points was the amount of time and effort it took to learn cloud technologies, and ultimately deciding on the most suitable infrastructure to host the ASR application on the cloud. My mentors also shared that the DevOps portion of their work took up a sizeable amount of their time. As such, I noted the importance of a quick turnaround time for deploying a new project. Furthermore, with the industry moving toward containerisation, I noted the importance of ease

of use of container orchestration software such as Kubernetes. As such, I decided on building a terminal tool to quickly spin up an Kubernetes cluster for a quick and easy deployment.

1.2 Objective

The main objective of this project was to speed up the turnaround time for deployment of new ASR applications. The main idea of this tool is to abstract away in-depth knowledge of cloud technologies from researchers, and allow them to readily and easily deploy their ASR applications to the cloud at the press of a button. This will allow researchers to then concentrate their efforts on the development of the ML model in the ASR application, and not have to worry about low-level cloud implementation details such as creating a security group role to allow ingress/egress traffic, or configuring their local environment to hit the cluster.

Drawing inspiration from minikube, which is a quick way to spin up a local cluster, for this FYP, I have decided to build cloudkube, which quickly spins up a Kubernetes cluster in the cloud. Users then have a simple barebones Kubernetes cluster hosted in the cloud where they can do dev testing.

The functional requirements for this project are detailed as follows:

a. `cloudkube` should provision infrastructure such as compute instances, storage, networking, and Kubernetes clusters in the cloud.

b. `cloudkube` should be easily extendable to different cloud providers.

c. `cloudkube` should create a Kubernetes environment, including deploying essential components like ingress controllers.

d. `cloudkube` should allow users to easily deploy containerized ASR applications to the Kubernetes cluster.

e. `cloudkube` should be able to deprovision and clean up resources without leaving anything dangling to prevent unnecessary costs.

f. `cloudkube` should maintain the state of provisioned infrastructure to avoid dangling resources in the cloud.

The non-functional requirements for this project are detailed as follows:

a. `cloudkube` should be easy to use, and have a gentle learning curve.

b. `cloudkube` should be user-friendly, with clear command structures and help options.

c. `cloudkube`, the CLI tool, should provision resources and deploy applications within a reasonable time frame.

d. `cloudkube` should provision and deprovision resources idempotently and atomically.

e. `cloudkube` should be able to work across different environments, including Windows, MacOS, and Linux.

f. `cloudkube` should support the addition of new features or integrations.

1.3 Project Scope

The scope of this project is focused on building a minimum viable project for cloudkube, to provision infrastructure for hosting applications in a Kubernetes environment in the cloud. cloudkube aims to simplify the development process by automatic infrastructure provisioning, Kubernetes cluster setup, cloud configuration, as well as application deployment. While the potential of cloudkube is massive, with possibilities for features such as multi-cloud support, advanced monitoring integrations, extensibility of modules and an infrastructure templates, the scope of this project has been deliberately constrained due to the time frame of this FYP.

Given the limited time frame, this project focuses on delivering core functionalities, which are as follows:

a. Efficiently spin up a Kubernetes cluster that applications can be easily deployed to.
b. Allow some customizability of the infrastructure:
    i. Architecture of compute instances.
    ii. Use of ingress controller.
    iii. Use of elastic file system for persistent volumes functionality in Kubernetes.
c. State tracking to ensure idempotent and consistent behavior.
d. Configuration of local environment to communicate with the Kubernetes cluster.
e. Clean deprovisioning and tear down of resources.

1.4 Contributions

This section highlights the key contributions made during the course of this project. The key contributions to this project can be roughly broken down into two parts. The first and main part relates to the infrastructure provisioning, which allows for any containerised workload to be deployed to a Kubernetes cluster hosted on the AWS cloud. The second part relates to optimizing the deployment of the ASR application.

Contributions toward Infrastructure Provisioning Workflow

a. **Optimized the workflow for creating Kubernetes clusters in the cloud:** In order to facilitate creating Kubernetes clusters in the cloud, I developed the `cloudkube` CLI tool to automate the provisioning of Kubernetes clusters, as well as their corresponding dependencies in the cloud..
b. **Creation of a user-friendly interface:** `cloudkube` was created with clear command structures and help options to ensure ease of use. As a tool aimed at developers who are not experts in cloud, `cloudkube` was built to abstract away in-depth cloud intricacies.
c. **Idempotency and Atomic Provisioning:** To align with the requirements stated above, `cloudkube` was also developed with state tracking, to ensure idempotent and atomic provisioning and deprovisioning of resources to prevent resource leaks and unnecessary costs.

d. **Easy user configurability:** After speaking to developers working on SpeechLab applications, and noting the use cases of different Kubernetes workloads, `cloudkube` provides customization options for infrastructure components such as compute instances, ingress controllers, and persistent volumes.

e. **Optimization of development workflow for authenticating local machine with Kubernetes cluster:** `cloudkube` also facilitates the configuration of local environments to communicate and authenticate seamlessly with the Kubernetes cluster.

f. **Extensive testing:** Extensive testing was also conducted to ensure the reliability and performance of the `cloudkube` tool. This included running multiple Kubernetes workloads against the provisioned cluster, which will be explained in detail in Chapter 4.

Contributions toward Optimizing ASR Kubernetes Deployment

a. **Optimized workflow for copying of models into compute worker instances:** The previous workflow for copying of the models into the EC2 worker instances was via secure copy `scp`. In this project, a more streamlined process was developed, and models were automatically pulled from cloud storage into compute instances [22].

1.5 Project Schedule

The project schedule is presented in Table 1-1. A Gantt chart for the project schedule is shown in Figure 1-1. It should be noted that these start and end dates serve as rough guidelines, especially for the implementation phase of the project. 1-1

Index	Task Name	Start Date	End Date
Preparation Phase			
1	Literature Review	31 May 2024	30 Jul 2024
2	Requirements Analysis	14 Jun 2024	15 Jul 2024
3	Requirements Refinement	15 Jul 2024	30 Jul 2024
4	System Design Development and Refinement	15 Jul 2024	30 Sept 2024
Implementation Phase			
5	Optimizing ASR Kubernetes Deployment	1 Oct 2024	7 Oct 2024
6	Provisioning Infrastructure for ASR	7 Oct 2024	9 Oct 2024
7	Development of main CLI interface of `cloudkube`	1 Oct 2024	31 Oct 2024
8	Ingress Controller Implementation	1 Nov 2024	15 Nov 2024
9	Persistent Volumes Implementation	15 Nov 2024	29 Nov 2024
10	`cloudkube` interactive setup wizard	15 Nov 2024	29 Nov 2024
11	`cloudkube` commands setup	2 Dec 2024	16 Dec 2024
Testing Phase			
12	ASR Kubernetes Workload	16 Dec 2024	15 Jan 2025
13	Multi Service Kubernetes Workload	16 Dec 2024	15 Jan 2025
Documentation Phase			
14	Project Proposal	12 Aug 2024	2 Sept 2024
15	Interim Report	16 Dec 2024	30 Jan 2025
16	Final Report	30 Jan 2025	24 Mar 2025
17	Presentation	24 Mar 2025	14 May 2025

Table 1-1: Project Schedule

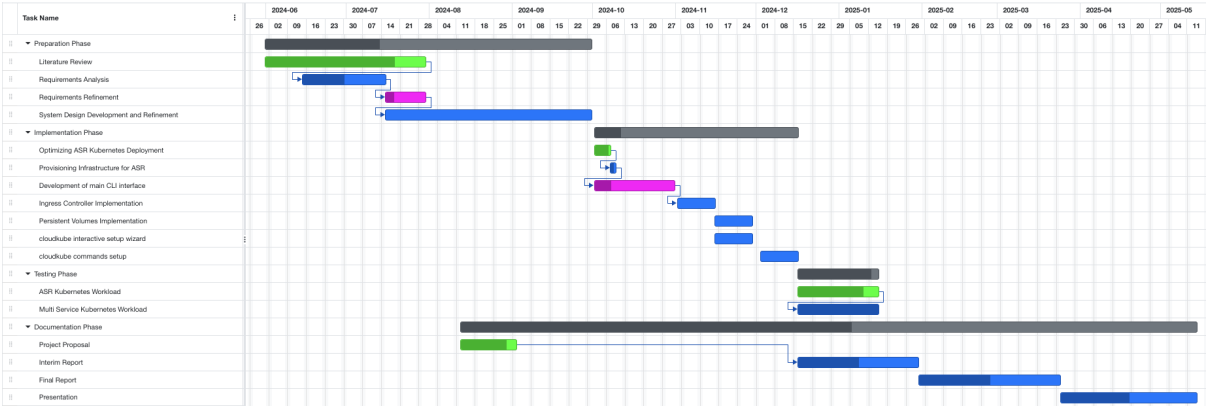

Figure 1-1: Gantt Chart for Project Schedule

1.6 Report Organisation

This report is structured into 5 different chapters, each detailing different aspects of the project.

Chapter 1 - Introduction: This chapter presents the background and introduction of the project. In this chapter, I elucidate some of the motivations for this project, and a brief overview on the intended outcome of this project.

Chapter 2 - Literature Review: This chapter reviews relevant technologies, and builds a foundational understanding for technical terminology used in future chapters. For completeness, this chapter will also discuss some of the pros and cons of each technology, which become relevant in later chapters.

Chapter 3 - Planning and Design: This chapter focuses on the analysis and design of the proposed solutions. In this chapter, I explore the current methods for handling containerised workloads, and show an optimized workflow for my proposed application.

Chapter 4 - Implementation: This chapter delves into the implementation of the proposed solutions. In this chapter, I detail the code and architecture used for `cloudkube`.

Chapter 5 - Results and Analysis: This chapter discusses the results of my proposed application, `cloudkube`. In this chapter, we will see the deployment of two applications to a Kubernetes environment provisioned by `cloudkube` - 1 ASR Application, and 2 Multi-service Kubernetes Application.

Chapter 6 - Conclusion and Future Work: This chapter presents the conclusions and recommendations for future work.

Literature Review

To determine the most optimal design and implementation tool, I conducted a comprehensive evaluation of the various cloud technologies. This involved exploring the capabilities of each cloud technology, and diving into the supporting documentation. By understanding the objectives, strengths, and weaknesses of each of the different technologies, I was able to narrow down on the problem that I was trying to solve, and select the best tools that will align with the requirements of the project.

2.1 Infrastructure as Code (IaC)

According to AWS, "Infrastructure as Code (IaC) is the ability to provision and support your computing infrastructure using code instead of manual processes and settings.". In the typical software development life cycle, when it gets to deploying applications, developers have to regularly set up, update, and maintain infrastructure in order to develop, test, and deploy applications [4].

With more and more applications coming up, and companies focusing on high velocity, it has become increasingly important for these manual tasks like infrastructure provisioning to be automated. Furthermore, manual infrastructure provisioning is also more error prone, especially when managing applications at scale (AWS, n.d.). As such, more and more IaC tools have come up, so that infrastructure can be provisioned reliably, quickly, and unambiguously to support our applications in the cloud. Next, we will discuss some of the common IaC tools, their uses cases, strengths as well as pitfalls.

2.1.1 Terraform

Perhaps one of the most popular IaC tools is Terraform. Terraform is an open-source IaC tool created by HashiCorp, which allows users to define and provision infrastructure declaratively. With Terraform, one can specify infrastructure resource in human-readable configuration files that can be reused, distributed, and modified [6].

Benefits

1. **Flexibility and Scalability:** Terraform integrates with over 1700 service providers governing different types of resources and services [6], which means that developers can automate the deployment and management of a wide variety of resources ranging from compute instances to storage solutions.

2. **Cloud Agnostic:** Terraform is cloud agnostic, meaning that a single configuration can manage multiple providers and handle cross-cloud dependencies.

3. **Open Source:** Terraform is open source and free to use.

4. **State Management:** Terraform manages its own state, meaning that cloud resources created and destroyed are all kept track of in a Terraform state file. This eliminates the worry of dangling or failed-to-provision resources.

How Terraform Works

1. **Providers:** At a high level, Terraform creates resources based on the targets exposed application programming interfaces (APIs). Communication with these APIs happens through providers, which are plugins that interact with various platforms and manage their resources. These providers can be seen as a logical abstraction of an upstream API. Today, Terraform supports over 1800 providers, including major cloud providers such as AWS and GCP, as well as providers for Docker, Kubernetes, and more [7]. Providers are normally defined in a top-level `terraform.tf` file as shown below:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.47.0"
    }
  }
  required_version = "~> 1.3"
}
```

2. **terraform plan:** This command creates an execution plan, which generates a preview of the changes that Terraform plans to make to your infrastructure [11]. By default, the following steps are taken:

   (a) Reads the current state of any already-existing remote objects to ensure that the Terraform state is up-to-date.

   (b) Compares the current configuration to the prior state and generates a difference between the current and prior state.

   (c) Proposes a set of changes that, if applied, will make the remote objects match the configuration.

3. **terraform apply:** This command executes the actions proposed in a Terraform plan [11]. By default, the following steps are taken:

   (a) All the steps as detailed in the above section on `terraform plan`.

   (b) Prompt for plan approval. For auto-approval, the flag `--auto-approve` can also be passed to the `terraform apply` command.

   (c) Undertakes the actions as indicated in the plan to modify existing cloud infrastructure to meet the desired current state.

   (d) Modifies the state file to match the current state.

4. **terraform destroy:** This command is used as a way to destroy all remote objects managed by a particular Terraform configuration [11].

5. **The Terraform State File:** To make all of the above possible, Terraform needs to maintain some kind of state, so that it is aware of the current state of resources on the cloud. This is done through the `terraform.tfstate` file, which is automatically generated/modified on every Terraform command that mutates state. For local development of Terraform, the `terraform.tfstate` file is generated locally. For production and other higher environments, it is standard practice for the `terraform.tfstate` file to be stored in the cloud, such as an S3 bucket, with DynamoDB for locking, to ensure effective version control [8].

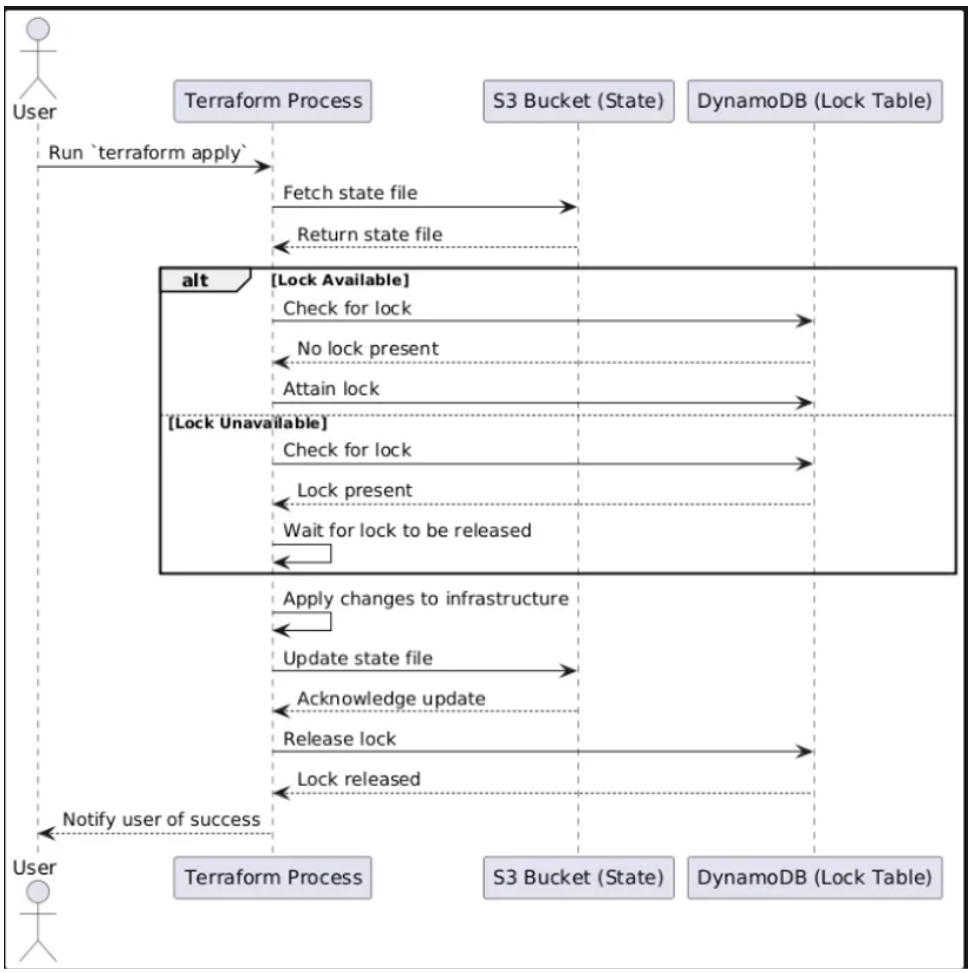

Figure 2-1: Sequence Diagram for Terraform Backend

The above diagram shows the typical workflow in handling terraform state files. In summary, state files are usually saved to some storage such as an S3 bucket. Access to these state files are then synchronized via a lock, which is commonly implemented in DynamoDB [8]. This allows race-free interaction with the provisioned infrastructure.

2.1.2 AWS CloudFormation

AWS CloudFormation is an IaC tools that allows one to manage infrastructure and automate any deployments using code [3]. CloudFormation is also done declaratively, and written in familiar languages such as YAML or JSON. This makes managing, scaling, and automating AWS resources fast and straightforward. However, the greatest downside is that CloudFormation is part of the AWS stack, and is not cloud agnostic. This means that developers are reliant on the AWS cloud should they choose to use AWS CloudFormation.

2.1.3 OpenTofu

OpenTofu is yet another open-source IaC tool designed to help with the provisioning and management of infrastructure [9]. It holds many similarities to Terraform as discussed above, but has an added feature of allowing for state encryption. There are also some differences in their licensing which will not be discussed here.

2.1.4 Ansible

Ansible is an open-source automation tool used for configuration management, application deployment, and task automation. Unlike Terraform and CloudFormation, which are declarative, Ansible uses an imperative approach to define the desired state of the infrastructure. The greatest upside of Ansible is that it is agentless, meaning it does not require any software to be installed on the target machines.

2.2 IaC Orchestration and Frameworks

IaC orchestration is used to enforce good programming practices, improving code readability and quality. This provides additional structure, automation, and improves maintainability for complex infrastructure. These tools provide a significant advantage over pure Terraform by improving code readability, promoting consistency across projects, and enabling faster collaboration and development in multi-environment setups.

Several IaC orchestration tools have also been built on top of terraform, such as terragrunt and terraspace. While an IaC orchestration tool will not be used in the implementation due to the main aim of keeping cloudkube portable and easily distributable, it is important to note how these tools fit into the whole cloud ecosystem.

2.2.1 Terragrunt

Terragrunt is a thin wrapper that is written on top of terraform, that provides some tooling and opinions [15]. Terragrunt also provides a way to keep your code DRY by using the generate helper method. Terragrunt also helps in the automated creation of backends, which can be a

chore if using raw terraform. Lastly, terragrunt allows for the injection of CLI hooks, allowing the utility to execute custom actions before or after terraform commands

2.2.2 Terraspace

Terraspace is a framework, which allows developer to dynamically generate terraform project in a centralized manner [16]. It is highly opinionated, and strongly enforces modularity and DRYness through stacks. On top of all the benefits offered by terragrunt, terraspace also allows the easy segregation of development and high-level environments through terraspace layering, which allows for the usage of the same code to deploy and create multiple environments.

2.3 Container and Container Orchestration

Containerisation has become the standard practice of the industry today. Allowing for quicker software development cycles, together with more efficient CI/CD practices and portability, containerisation provides us a powerful, and agile way to manage our applications [20]. Along with the growing complexity of distributed systems, container orchestration software such as Kubernetes are also used to automate deployment, handle scalability and fault tolerance, as well as efficient resource utilization. This chapter will discuss some of the common container and container orchestration software, which are pertinent to this project given that most SpeechLab applications have been containerised for the cloud

2.3.1 Docker

Docker is a containerisation software which aids in development, shipping, and deployment. Docker enables the separation of application from infrastructure so that software can be delivered quickly and reliably [1]. With docker, code, as well as version-locked dependencies are baked into a container image. This means that anyone building from this image on any machine in the world, regardless of architecture, will build the same thing, and run the same application. This creates reproducible environments of an application based on a single Docker Image.

2.3.2 Kubernetes

Kubernetes is an open-source container orchestration software that helps to automate deployment, scale, and manage containerised applications. Kubernetes provides a framework to run distributed systems resiliently, taking care of scaling and failover of the application [13].

Key Features of Kuberentes

1. **Self-healing:** Kubernetes restarts, replaces, or kills containers that are unhealthy, and doesn't advertise these containers to clients until they are ready to serve.

2. **Storage Orchestration:** Kubernetes allows the automatic mounting of a storage system of the developers choice.

3. **Horizontal Scaling:** Kubernetes can scale up or down an application with a simple command, UI, or automatically based on user-defined parameters.

4. **Service Discovery and Load Balancing:** Kubernetes can expose a container using the DNS name of the container, or using its own IP address. Kubernetes can also load balance and distribute network traffic to ensure deployment stability.

Kubernetes Architecture

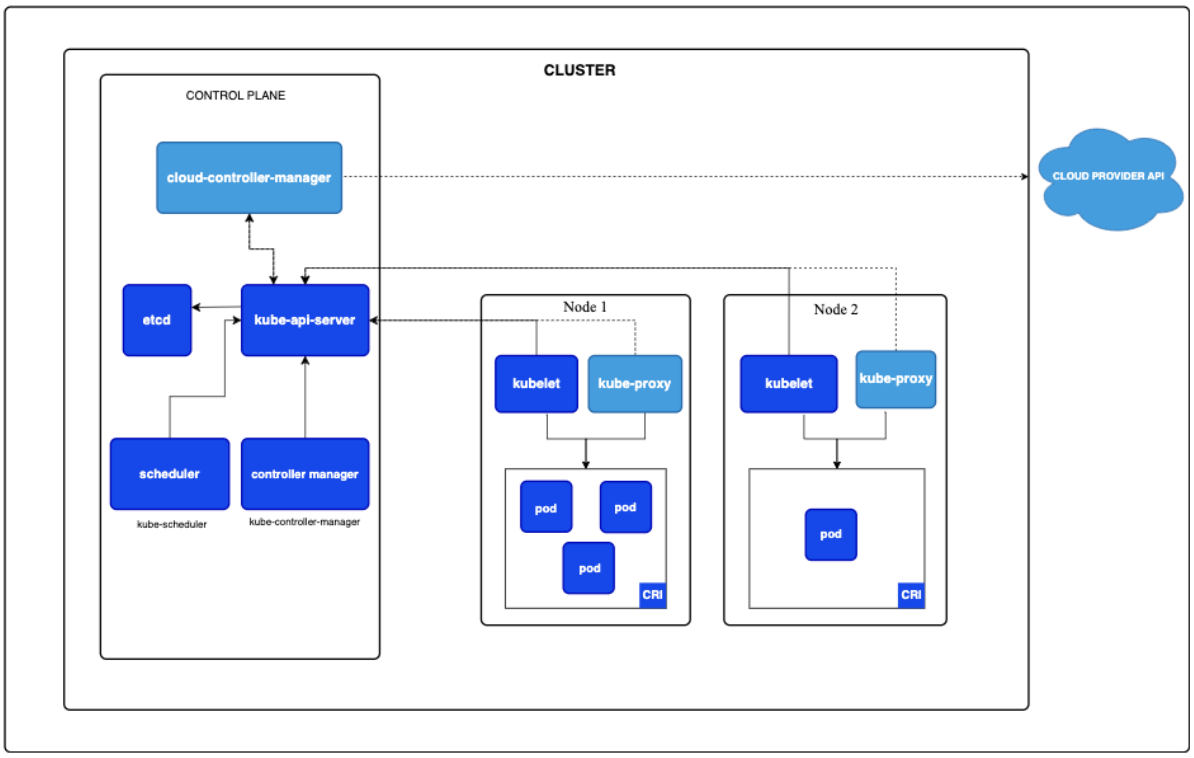

Figure 2-2: Typical Kubernetes Architecture

A Kubernetes cluster typically comprises the Control Plane as well as multiple Worker Nodes. Each of these nodes hold different responsibilities, which are detailed below.

1. **Master Node**

   (a) The Kubernetes master node, also known as the control plane, is the controller of the whole system. This is where all operational decisions are made, including scheduling, balancing, and responding to events.

   (b) *kube-api-server:* The Kubernetes API Server is the main module within the master node which receives all REST requests for modifications, thereby serving as the frontend to the cluster [5]. It acts as an interface for interaction with the cluster.

   (c) *kube-controller-manager:* The kube-controller-manager runs a distict number of daemon processes, which help to regulate the shared state of the cluster, as well

as perform routine tasks. When changes in configurations occur, the kcm is responsible for spotting the change and working toward the new desired state [5].

(d) *Kube-scheduler:* The Kubernetes scheduler helps to schedule the pods on the various nodes based on resource utilization [5]. The kube-scheduler is aware of all the resources available, and is responsible for allocating pods to nodes in an efficient manner to avoid resource exhaustion. Should no resources be available, the kube-scheduler puts pods in a pending state until resources are available again.

2. **Worker Nodes**

   (a) The Kubernetes worker node, is a worker machine in the Kubernetes cluster. There can be multiple worker nodes in a Kubernetes cluster. Each node is responsible for running pods, and is managed by the master node.

   (b) *kubelet:* The kubelet is the main service on a node, which regularly takes in pod specifications through the kube-api-server. The Kubelet is responsible for ensuring that pods and containers are healthy, and running in the desired state [5]. The Kubelet is also responsible for health checks.

   (c) *kube-proxy*: The kube-proxy is a service that runs on each worker node, that deals with networking. It is responsible for exposing services to the external world, or to the rest of the cluster. It performs request forwarding to the correct pods across various isolated networks in a cluster [5].

   (d) *Container runtime:* The container runtime is responsible for pulling images from a container image registry (i.e. Dockerhub, Amazon AWS ECR), and starting and stopping containers. The most common plugin used for this us Docker [5].

3. **Pods**

   (a) A pod is the smallest unit that can be scheduled in Kubernetes [5]. Pods help in scalability through replication. In the case of errors or failed health checks, Kubernetes also creates new pods, and destroys unhealthy ones. A single pod can contain multiple containers, and these containers share an IP address and port space, and are able to communicate internally via localhost.

Networking in Kubernetes

Another concept relevant to the development of `cloudkube` is networking in Kubernetes. The following section will two main networking concepts in Kubernetes, which will become relevant to our implementation in Chapter 4.

1. **Ingress:** A Kubernetes ingress object is an API object that manages external access to services in a cluster, typically through HTTP/HTTPS. The Ingress object allows us to map traffic to different backends based on rules defined via the Kubernetes API.

2. **Ingress Controller:** a. In order for an Ingress to work in the cluster, there must be an ingress controller running. Ingress controllers are not started automatically with a cluster, and requires a one to be specified and provisioned for ingress resources to work.

Storage in Kubernetes Lastly, this section will detail how kubernetes handles ephemeral and persistent storage. Similarly, this becomes relevant to our implementation as detailed in Chapter 4.

1. **Volumes:** Volumes in Kubernetes provide a way for containers in a pod to access and share data via the filesystem. This can be used for data sharing between two different containers in the same pod, or even between two different pods. There are two broad categories of volumes. Ephemeral volume types have the lifetime of a pod, and follow the lifecycle of a pod, while persistent volumes exist beyond the lifetime of a pod.
2. **Persistent Volumes (PV):** A persistent volume is a piece of storage in the cluster that has been provisioned by the developer. The data in the PV remains even after pods are destroyed or recreated.
3. **Persistent Volume Claims (PVC):** A Persistent Volume Claim is a request for storage by a user. PVCs consume PV resources, and have defined access modes, such as ReadWriteOnce, ReadOnlyMany, ReadWriteMany, or ReadWriteOncePod.

2.4 Infrastructure and Cloud

This chapter details some of the key cloud infrastructure that is provisioned in order to host speech applications. For a proof of concept, and due to limited access to various clouds, everything in this paper is hosted on the AWS cloud. However, it should be noted that many of these concepts (such as VPC, subnets, load balancers etc.) are similar in other clouds, and cloudkube is extendable to other cloud providers as well. Each subsection will discuss the main uses of each of these components, and briefly detail the corresponding names of these components in AWS, Microsoft Azure Cloud, as well as Google Cloud Provider.

2.4.1 Virtual Private Cloud (VPC)

A virtual private cloud allows the creation of logically isolated sections of the cloud where different resources can be launched in a user-defined virtual network. Developers have full control over the virtual networking environment, including the selection of IP address ranges, creation of subnets, as well as configuration of route tables and network gateways. There will be mentions of some VPC related terminology in the following chapters, all of which are defined here.

- **Subnets:** Subnets allow the division of the VPC into different IP address ranges. They can be public or private.
- **Route Tables:** Route Tables allow the control of routing of traffic within the network. This helps in directing traffic between subnets, or between different VPCs.
- **Internet Gateway (IGW):** The Internet Gateway can be attached to a VPC to allow communication between resources in the VPC and the internet.

- **NAT Gateway:** The Network Address Translation (NAT) Gateway enables instances in a private subnet to connect to the internet or other cloud services while preventing the internet from initiating a connection with those instances.
- **Security Groups / Network Access Control Lists (ACLs):** These are virtual firewalls that control ingress and egress traffic at the instance and subnet levels, providing granular security controls.

Cloud Provider VPCs

- AWS Cloud: AWS Virtual Private Cloud
- GCP: Virtual Private Cloud Network
- Microsoft Azure: Virtual Networks

2.4.2 Subnets

A subnet is a range of IP addresses in a VPC. Subnets allow us to segment the network inside the VPC to group resources and control their access to each other and to the outside world. A subnet can be thought of as a sub-network within a VPC, where resources like EC2 instances can reside.

Key Features of Subnets

1. Public and Private Subnets
    - Public Subnets are subnets with a route to an IGW. Instances within this subnet can directly communicate with the internet, provided that they have a public IP address or an Elastic IP.
    - Private subnets are subnets which do not have a route to an IGW. Instances in a private subnet cannot communicate with the internet directly. They can only access the internet via a NAT Gateway, or NAT instances in a public subnet.
2. CIDR Blocks (Classless Interdomain Routing)
    - Each subnet is associated with a CIDR block that defines its IP address range. CIDR blocks of subnets are always within (longer prefix mask) the CIDR block of the VPC.
3. Availability Zones (AZ)
    - Subnets are created within a specific AZ in a region. Resources within a subnet are guaranteed to be located within the same AZ.
    - Distribution of subnets across multiple AZs help to increase availability and fault tolerance.
4. NACLs
    - Subnets are protected using NACLs, which are stateless firewalls that operate at the subnet level. They control ingress and egress traffic to and from the subnets based on user-defined rules.

2.4.3 Virtual Machines

Kubernetes applications that run in the cloud require compute to be done on virtual machines. Virtual machines are web services offered by cloud providers which allow users to run virtual servers in the cloud. The virtual machines are normally configurable in terms of the choice of processor, storage, networking as well as operating systems.

Cloud Provider Virtual Machines

- AWS Cloud: AWS Elastic Compute Cloud (EC2)
- GCP: Google Compute Engine (GCE)
- Microsoft Azure: Azure Virtual Machine (VM)

Pertinent to this paper, EC2 instances will be provisioned as nodes in the Kubernetes cluster. The Kubernetes pods are then ultimately run on these nodes.

2.4.4 Security Groups

Security groups are virtual firewalls that control inbound and outbound traffic to and from resources. Security groups are stateful, meaning that if you allow an incoming request from a particular IP address and port, the response is automatically allowed without needing a separate outbound rule. Some cloud providers such as Azure also draw a distinction between security groups which operate at the application layer (operating on compute instances and services), as well as the network layer (operating on ingress and egress IP addresses and protocols).

2.4.5 Elastic Network Interfaces (ENI)

An ENI is a logical networking component in a VPC that represents a virtual network card. These ENIs are attached to instances launched in the same AZ. Every VM instance has a primary ENI attached by default, providing the instance with networking capabilities. These ENIs have multiple use cases. Pertinent to this paper, the ENIs are used to facilitate communication between the Kubernetes control plane and worker nodes within the cluster.

2.4.6 Instance Templates

Instance Templates allow users to define the configuration for launch VM instances in a more flexible and reusable way. They allow the specification of instance parameters such as instance type, key pair, security groups and more. These templates can then be used by Auto Scaling Groups, Spot Fleets, and other services to provision VM instances according to the configurations specified.

Cloud Provider Instance Templates

- AWS Cloud: AWS Launch Template

- GCP: Instance Templates

Appendix A shows an example AWS Launch Template to provision a VPC for use with Kubernetes.

2.4.7 Managed Kubernetes Service

Managed Kubernetes Services are services offered by cloud providers which can be used to run Kubernetes without needing to install, operate, and maintain your own Kubernetes control plane or nodes. The managed Kubernetes service is able to automatically scale control plane instances based on load, detect and replace unhealthy control plane instances, and provide automated version updates and patching for them. These managed service also often offer easy integrability with other cloud services such as file systems for persistent volume storage, and load balancing for load distribution.

Cloud Provider Managed Kubernetes Service

- AWS Cloud: Elastic Kubernetes Service (EKS)
- GCP: Google Kubernetes Engine (GKE)
- Microsoft Azure: Azure Kubernetes Service (AKS)

2.4.8 File Systems

File systems provide a simple, serverless, set-and-forget elastic file system for use with cloud resources or on premise infrastructure. They are built to scale on demand to petabytes without disrupting applications, growing and shrinking automatically as files are added or removed, thereby eliminating the need to provision and manage capacity to accommodate growth.

Cloud Provider File Systems

- AWS Cloud: Elastic File System (EFS)
- GCP: Cloud Filestore
- Microsoft Azure: Fileshare

2.4.9 Scaling Groups

Often times, it is important for compute instances to automatically scale with load. Scaling groups are a collection of compute instances treated as a logical grouping. Its main purpose and objective is to provide for automatic scaling and management. This allows the user to maintain the number of instances in a scaling group. The scaling group starts by launching enough instances to meet its desired capacity. It then maintains this number of instances by conducting periodic health checks on instances in the group. If an instance becomes unhealthy, the scaling group then terminates the instance and launches another instance to replace it.

Cloud Provider Scaling Groups

- AWS Cloud: Auto Scaling Groups (ASG)
- GCP: Managed Instance Groups (MIG)
- Microsoft Azure: Azure Virtual Machine Scale Sets (VMSS)

2.4.10 minikube

minikube is a way for one to quickly set up a local Kubernetes cluster. It is compatible with macOS, Linux and Windows. The main goal of minikube is to help application developers do quick testing of their Kubernetes deployments locally, as well as a tool used for learning by new Kubernetes users [14]. By default, minikube creates a one-node cluster locally, and supports addons such as ingress controllers. As we will see in the following chapter, cloudkube functionality closely mirrors that of minikube, supporting the objective of cloudkube which is to provide a quick and easy testing environment for Kubernetes developers in the cloud.

Planning and Design

With the rapid adoption of containerisation technologies, organizations are increasingly turning to containers for their scalability, portability, and efficiency in deploying modern applications [10]. This shift has prompted cloud providers to develop managed container solutions, simplifying the orchestration of containerized workloads.

In this chapter, we will detail the current approach taken in deploying a containerized application to the cloud platform. We will then analyse some of its pitfalls, and show how the proposed solution, `cloudkube`, aims to solve, and alleviate some of these issues. Mainly, each of these subsections will detail the number of steps it takes to get a working container management system up and running, as well as the amount of configuration required. We will then see the much quicker development workflow that `cloudkube` offers.

3.1 Traditional Approaches

Within the AWS ecosystem, there are two primary services offering container orchestration solutions. They are the AWS Elastic Container Service (ECS), as well as a managed Kubernetes service, the AWS Elastic Kubernetes Service (EKS). With 96% of companies who use containerisation in production opting for Kubernetes for container orchestration [21], this report will mainly focus on deploying a containerized workload on Kubernetes. That said, AWS ECS will be included in our analysis for completeness.

3.1.1 AWS Elastic Container Service (ECS)

AWS ECS is a fully managed container orchestration service that helps you to deploy, manage, and scale containerised applications. AWS ECS allows for the running and scaling of container workloads across AWS regions in the cloud, without the complexity of managing a control plane [2].

Workflow for Deploying a containerized application to ECS

1. Create a Virtual Private Cloud (VPC)
    (a) VPCs are used to launch AWS resources into a user-defined virtual network. They serve as the foundational network infrastructure to deploy cloud resources. In the creation of our VPC, multiple factors need to be considered, including the CIDR blocks, subnet configurations, as well as Availability Zones.
    (b) By default, the VPC supports IPv4 traffic, though IPv6 traffic can be optionally enabled. Users must also decide on tenancy options, which dictate whether

resources share hardware with other accounts or if their resources have dedicated access to hardware.

(c) Subnets, essential for segmenting resources, must also be carefully designed. Public subnets provide direct access to the public internet, and must be configured with an Internet Gateway. On the other hand, private subnets are typically used to access and host internal resources such as databases, and must be configured with a NAT or Egress only Internet Gateways in order to access the public internet.

2. Configure Security groups and network ACLs

   (a) Ensuring robust security control is essential when deploying containerised applications. AWS offers two access control mechanisms, namely Security Groups, and Access Control Lists.

   (b) Security Groups operate at the resource level, managing inbound and outbound traffic for associated resources, such as EC2 instances. They operate at the instance level, and is stateful, meaning that all return traffic is allowed, regardless of rules. If container instances are launched in multiple regions, then developers need to create a security group in each region, and associate it with the corresponding resource.

   (c) On the other hand, Network Access Control Lists allow or deny inbound and outbound traffic at the subnet / network level. These can be used as an additional layer of security, and are stateless, meaning that return traffic must be explicitly allowed by the rules.

3. Decide between Fargate (Serverless) or EC2 Instances

   (a) Developers also need to make an important decision between using AWS Fargate, a serverless option, or EC2 instances, for compute. AWS Fargate allows a way for containers to be run directly, without any virtual machines, or servers to think about. AWS Fargate is a much more managed solution, and automatically manages upgrades and out of date dependencies, but are slightly more expensive than EC2 instances at high utilization [18]. On the other hand, EC2 instances allow the system administration more control over the instances in the cluster. This means that the system administrator has more oversight on maintaining the cluster and optimizing it.

4. Preparing and pushing the container image

   (a) AWS ECS task definitions used Docker images to launch containers on the container instances in the cluster. Assuming we have a ready and runnable image on our local computer, we need to push this image to a container registry. This can be something like AWS Elastic Container Registry (ECR), or Dockerhub.

5. Create a task

   (a) Create a task definition, which specifies the Docker image to use for the containers, how many containers to use in a task, as well as the resource allocation for each container.

   (b) Next, we need to create a service using the task definition, and attach the security group created previously to this service. It is only at this point, with much manual configuration, that our containerised application is deployed to AWS ECS.

3.1.2 AWS Elastic Kubernetes Service (EKS)

With companies moving toward Kubernetes to handle increasingly complex workloads, this paper focuses on deployment of a containerised workload to Kubernetes. AWS EKS is a fully managed Kubernetes service that enables Kubernetes workloads to be run seamlessly on AWS cloud.

Workflow for deploying a containerised application to EKS

1. Creating a VPC
    (a) As mentioned previously, the VPC serves as the foundational network infrastructure that we deploy our cloud resources to. However, the VPC for use with EKS must meet EKS networking requirements in order to ensure proper communication between Kubernetes nodes, as well as between the nodes and the external internet. These requirements include IP addressing requirements, DNS resolution support, and subnet configuration, just to name a few. The major list of requirements are detailed in Appendix B. In order to minimise configuration for developers, AWS has also provided a pre-defined CloudFormation template (Appendix A). However, this template is still very complex, and difficult to find amongst all the AWS developer documentation, making this process an extremely tedious one.
2. Create the EKS
    (a) The EKS can be created via AWS CLI or the console. Briefly, through the setup wizard, we will need to do the following:
        i. Decide on a name of the cluster, and Kubernetes version to be used in the cluster.
        ii. Create and use a Cluster IAM role, which allows the Kubernetes control plane to manage AWS resources on your behalf.
        iii. Create and use a Node IAM role, which will be attached to the EC2 instance launched and registered with the cluster.
        iv. Select the VPC created earlier for use with the EKS cluster resources.
        v. Select the subnets in the VPC created earlier where the control plane may place elastic network interfaces (ENIs) to facilitate communication with the cluster. These would typically be the private subnets created earlier in the creation of the VPC (one per availability zone).
        vi. Choose from a wide selection of AWS add-ons (Appendix C). This is required for configuring ingress controllers, integrating with external file systems, etc.
3. Configure the EKS Node Groups
    (a) Kubernetes works by scheduling pods onto nodes, which are our compute instances. AWS EKS uses EC2 instances with Auto Scaling Groups to satisfy these requirements. At this stage, we would need to choose from the different Amazon Machine Image (AMI) types, Instance Types, as well as determine the scaling configuration for the node group, and set up networking between node groups.
4. Configure our local environment to communicate with the EKS Cluster

(a) As with every Kubernetes cluster, the local environment needs to attain certificates, and other metadata to apply the Kubernetes config to the cloud environment. In typical production environments, this is handled through git CI/CD pipelines, or a CI/CD orchestrator tool such as Jenkins. In development, to configure the kubeconfig locally, we can run `aws eks --region <region> update-kubeconfig --name <name>` to configure our local configuration so that we are able to communicate with the Kubernetes cluster.

5. Run our Kubernetes application

   (a) Assuming that we have a Kubernetes configuration ready to go, we can then apply the Kubernetes configuration to the cluster, and begin testing our application.

3.2 Challenges

As illustrated above, the traditional approach of deploying a containerised application is highly cumbersome and error-prone. Missing any of the essential steps listed often leads to hours of debugging – whether it's identifying why the website is unresponsive, troubleshooting connectivity issues between nodes or exposing the application to the internet. Each and every step plays a critical role in ensuring that the application runs smoothly and error free. The traditional approaches are also very time-consuming, and have a steep learning curve. Engineers must gain mastery of the different cloud technologies, and know how to integrate them seamlessly, making this task both challenging, and resource-intensive.

Complexity of Configuration

Setting up the environment for containerised application deployment requires a lengthy process of setting up networking within the cluster. This requires developers to understand and gain a mastery over the different cloud components, and learn to apply them within the cloud. A small misconfiguration here could lead to connectivity issues, or resource exhaustion. Furthermore, configuring the security layer also requires careful planning, on order to ensure both security and functionality, making the process extremely time consuming and tedious.

Manual Work

Administrators are required to perform many repetitive tasks, especially if they are required to provision infrastructure for multiple applications. This requires the repeated configuration, and drastically slows down the deployment process and introduces inefficiencies.

Risk of Errors

Given the numerous configuration options and types, errors in any of the above steps can also lead to failures, downtime, or suboptimal performance of the cluster.

Lack of Streamline Integration

The approaches detailed above are also only used for a barebones configuration of a cluster, and does not integrate other services such as Load Balancing or persistent storage. Integration

with these services, should an application require it, will take additional manual setup, and maintenance.

Steep Learning Curve

Provisioning of these infrastructure resources require a deep understanding of container orchestration and services. Without prior expertise, it is easy to misconfigure the cluster, which could lead to security vulnerabilities or broken deployments. Learning the different cloud technologies is very time consuming, and will inevitably result in reduced velocity in teams.

3.3 Proposed Solution

The proposed solution, `cloudkube`, aims to alleviate the above challenges. Besides having an intuitive user interface, `cloudkube` leverages on the strengths of existing IaC frameworks such as terraform. This allows users to abstract in-depth knowledge about terraform and the cloud away, and provides a quick and easy way to deploy a containerised application to the cloud.

3.3.1 Key Principles

`cloudkube` is built on three main principles:

1. Simple and Minimal Dependencies: In order to satisfy the simplicity of packaging, distribution, and installation of `cloudkube`, we opt to keep dependencies as lean as possible, to account for compatibility with host architecture as well as conflicting dependencies. The distribution is also kept lightweight, in that `cloudkube` is not meant for a full-blown production setup, but rather a quick and easy way to do development testing in the cloud. As such, more robust frameworks such as terraspace and terragrunt are intentionally excluded. With these, and python installed on a user's machine, a user is able to do a simple 'pip install `cloudkube`' to install `cloudkube` on to their system, and get their first Kubernetes application running in the cloud.

2. Leveraging current IaC best practices and tooling: `cloudkube` is opinionated in the sense that it defines a terraform configuration of the cloud in a specified manner. Used as a tool for beginners, terraform details and implementation are abstracted away, and `cloudkube` sets up a Kubernetes cluster in a pre-defined manner, much like what is done in minikube. `cloudkube` also uses the existing methods of tracking terraform state (through the '.tfstate' file). On top of that, to track the installation state of dependencies such as ingress-controllers or persistent volume infrastructure, a configuration file is introduced as well.

3. Intuitiveness and Ease of Use: Drawing inspirations from some of the best-in-class frameworks used today, `cloudkube` is built with a easy to use terminal interface. From create-react-app, `cloudkube` implements a setup wizard where users respond to prompts to customize their cluster to fit their development needs. From minikube, cloudkube implements imperative methods to configure cluster ingress controllers and persistent volumes.

3.3.2 Workflow

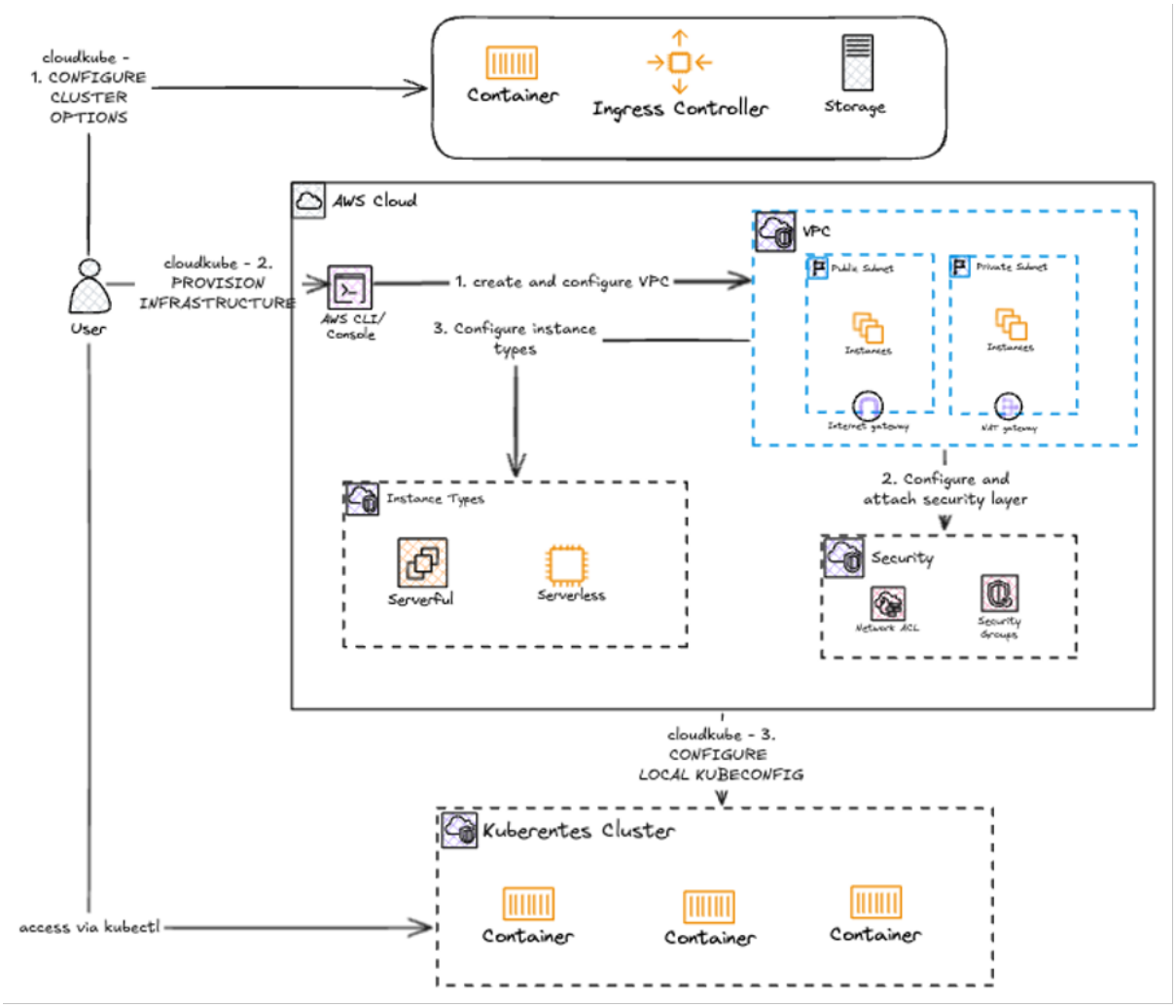

Figure 3-1: Workflow for creating a Kubernetes cluster with `cloudkube`

The workflow to get an up and running Kubernetes cluster on the cloud is made much more effortless with `cloudkube`. Besides having to wait for about 10 minutes for the cluster to be provisioned, as well as some configurations for authenticating with the cloud, `cloudkube` automates the provisioning of all cloud resources at the click of a button.

Typical Workflow for Deploying an Application `cloudkube`

1. Packaging and Containerisation

   (a) As with deploying any application to the cloud with Kubernetes, the first step to deploy and application with `cloudkube` is to containerise the application. This can be done with Docker.

   (b) Container images to be used in Kubernetes pods should also be pushed to a container registry which the cloud is able to pull from. Examples include Amazon's Elastic Container Registry, or Dockerhub. Kubernetes secrets for authenticating with these image registries should also be appropriately setup.

2. Defining the `cloudkube` configuration

    (a) `cloudkube` relies on a configuration file 'config.json' in its root directory to conditionally provision different types of infrastructure. Depending on the use-case, as well as the requirements of the application, users can select from different computer architecture types, availability zones as well as naming conventions for resources to be created in the cloud.

    (b) Based on the the Kubernetes configuration, users also need to decide whether to provision and ingress controller, as well as what type of ingress controller to provision.

    (c) Lastly, `cloudkube` realises PersistentVolumes in Kubernetes through provisioning an Elastic File System which can be mounted to Kubernetes nodes. This helps to provide persistent storage for Kubernetes deployed applications.

3. Provision Infrastructure

    (a) Before the cloud infrastructure can be applied, the root Terraform module needs to be initialised. `cloudkube` automatically initializes Terraform, and downloads the relevant providers and modules for its inbuilt Terraform configuration.

    (b) The next step taken by `cloudkube` is to plan the infrastructure through calling 'terraform plan'. This creates a plan for the infrastructure to be created in the cloud, and `cloudkube` then outputs the plan, and seeks user approval for creating these resources.

    (c) Upon confirmation by the user, `cloudkube` applies these changes to the cloud, and creates a Kubernetes cluster in the cloud. The infrastructure is created based on the user-specified configuration in the previous step.

4. Configure Local Machine to Communicate with the Kubernetes Cluster

    (a) In typical Kubernetes workflows, users would have to run some cloud-provider specific commands to populate their local Kubernetes configuration in order to communicate with the Kubernetes cluster.

    (b) This step is automatically done by `cloudkube` within the provisioning step. This means that users are able to get straight to applying their application to the Kubernetes cluster.

5. Apply application to Kubernetes Cluster

    (a) Users can apply their Kubernetes configuration to the configured Kubernetes cluster, and begin interacting with their application. Users are also able to interact with their cluster through the various kubectl commands.

Step by step instructions for the initial setup, as well as for developing with `cloudkube`, are detailed in Appendix D.

As we can see from the above workflow, the chore and tedium of infrastructure provisioning is greatly abstracted away from the user. This allows a user to deploy their containerised application to the cloud in a matter of minutes. Specific to deploying the ASR application, the use of `cloudkube` abstracts away many tedious steps as illustrated by Song Yu's work on

Infrastructure as Code with Terraform and Terragrunt. More specifically, with `cloudkube`, users do not need to do the following steps:

1. **Initialise Terraform:** `cloudkube` does this behind the scenes for you. Fetching modules, providers, and updating backend configuration are all done automatically by `cloudkube`, through `terraform init`.
2. **Local Kubernetes Context Configuration:** After the infrastructure provisioning stage, `cloudkube` automatically connects to the cluster, and updates your local Kubernetes configuration files. This ensures that the user is able to communicate with the Kubernetes cluster via any `kubectl` commands.
3. **Mounting the EFS on Amazon EC2 Instance:** `cloudkube` automatically mounts an EFS to the Kubernetes nodes in the EKS. After applying the Kubernetes configuration to the cluster, users are able to interact with the file system.

As illustrated, the deployment process of ASR application to the cloud is greatly simplified. Furthermore, this is just an example of a deployment of an ASR application. In the implementation section, we will also see another example of deploying a more complex application with multiple pods, and services that communicate with each other.

With `cloudkube`, development times are greatly decreased, and developers can focus on the application business logic rather than having to debug an obscure permission setting on the cloud, or be a master of cloud technologies. Furthermore, users are also accorded configurability of the cluster, including being able to choose their machine architecture, ingress controller, as well as the provisioning of an EFS volume for PersistentVolume objects in Kubernetes. The configuration of `cloudkube` is also a low-cost one, in that low-spec machines are chosen from the various machine images offered by AWS.

3.3.3 Benefits of `cloudkube`

By leveraging the power of IaC, `cloudkube` is able to maintain many of the guarantees and benefits that terraform provides. `cloudkube` also greatly simplifies and accelerates the provisioning of infrastructure.

High Customizability and Extensibility

In order to meet the needs to different Kubernetes workloads, `cloudkube` is built to be configurable. For example, where Ingress resources are declared in Kubernetes, `cloudkube` offers the installation of a nginx ingress controller to satisfy Kubernetes Ingress resources. Similarly, in order to satisfy PersistentVolume and PersistentVolumeClaim resources, `cloudkube` also offers the installation of an EFS in the cluster. On top of this, users are also able to customize the machine architecture to ensure compatibility with their applications.

Low Cost Prototyping Solution

`cloudkube` is configured with mostly AWS Free Tier resources. As such, creating a Kubernetes cluster with `cloudkube` is cheap, and not resource intensive. Furthermore, `cloudkube` only

provisions what you want. This means that only resources that are required by the Kubernetes application are provisioned. This ensures infrastructure is kept lean, and costs are kept low.

Lean Package Distribution on pyPI

`cloudkube` is conveniently distributed on pyPI, which means that anyone with python and pip installed on their machine and conveniently download `cloudkube` with a single command 'pip install `cloudkube`'.

Preconfigured Permissions, Security Groups, Access Control Lists, IAM Roles

`cloudkube` comes with pre-configured permissions, security groups, ACLs, and IAM Roles. Configured IAM Roles ensures that cluster nodes have sufficient permissions on AWS to communicate with each other as well as the external internet. The pre-configured security groups also ensure that traffic between different cloud components is properly configured, to allow flow of information between these components securely. Ingress and Egress resources also govern traffic into and out of the cluster, ensuring that users can reach their deployed services.

Built on terraform

Being one of the leaders in IaC tooling, terraform is widely used across the industry. It is both robust, and battle-tested by leading firms in the industry. `cloudkube`, being built on terraform, retains all of the benefits that terraform brings. The most notable would perhaps be the tracking of the state file (`terraform.tfstate`). Terraform uses the state to determine the changes to be made to infrastructure. This ensures that the state of the cloud is always reflected on the terraform state file. Prior to any provisioning or teardown commands in `cloudkube`, the existing cloud infrastructure is compared against the state file, and `cloudkube` fails safely if the system is in an inconsistent state. This avoids leaving dangling resources on the cloud.

Implementation

Updates to this chapter will be included in the final report.

Results and Analysis

Updates to this chapter will be included in the final report.

Conclusion and Future Work

Updates to this chapter will be included in the final report.

Appendix A: AWS CloudFormation YAML Template for EKS IPv4

The following is the YAML template used for provisioning the infrastructure for an EKS compliant VPC (IPv4):

```
---
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Amazon EKS Sample VPC - Private and Public subnets'

Parameters:

  VpcBlock:
    Type: String
    Default: 192.168.0.0/16
    Description: The CIDR range for the VPC. This should be a valid private (RFC
        1918) CIDR range.

  PublicSubnet01Block:
    Type: String
    Default: 192.168.0.0/18
    Description: CidrBlock for public subnet 01 within the VPC

  PublicSubnet02Block:
    Type: String
    Default: 192.168.64.0/18
    Description: CidrBlock for public subnet 02 within the VPC

  PrivateSubnet01Block:
    Type: String
    Default: 192.168.128.0/18
    Description: CidrBlock for private subnet 01 within the VPC

  PrivateSubnet02Block:
    Type: String
    Default: 192.168.192.0/18
    Description: CidrBlock for private subnet 02 within the VPC

Metadata:
  AWS::CloudFormation::Interface:
    ParameterGroups:
      -
        Label:
          default: "Worker Network Configuration"
        Parameters:
          - VpcBlock
          - PublicSubnet01Block
          - PublicSubnet02Block
          - PrivateSubnet01Block
          - PrivateSubnet02Block

Resources:
  VPC:
    Type: AWS::EC2::VPC
    Properties:
```

```
      CidrBlock:  !Ref VpcBlock
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
      - Key: Name
        Value: !Sub '${AWS::StackName}-VPC'

  InternetGateway:
    Type: "AWS::EC2::InternetGateway"

  VPCGatewayAttachment:
    Type: "AWS::EC2::VPCGatewayAttachment"
    Properties:
      InternetGatewayId: !Ref InternetGateway
      VpcId: !Ref VPC

  PublicRouteTable:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VPC
      Tags:
      - Key: Name
        Value: Public Subnets
      - Key: Network
        Value: Public

  PrivateRouteTable01:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VPC
      Tags:
      - Key: Name
        Value: Private Subnet AZ1
      - Key: Network
        Value: Private01

  PrivateRouteTable02:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VPC
      Tags:
      - Key: Name
        Value: Private Subnet AZ2
      - Key: Network
        Value: Private02

  PublicRoute:
    DependsOn: VPCGatewayAttachment
    Type: AWS::EC2::Route
    Properties:
      RouteTableId: !Ref PublicRouteTable
      DestinationCidrBlock: 0.0.0.0/0
      GatewayId: !Ref InternetGateway

  PrivateRoute01:
    DependsOn:
    - VPCGatewayAttachment
    - NatGateway01
    Type: AWS::EC2::Route
    Properties:
      RouteTableId: !Ref PrivateRouteTable01
      DestinationCidrBlock: 0.0.0.0/0
      NatGatewayId: !Ref NatGateway01
```

```
112
113       PrivateRoute02:
114         DependsOn:
115         - VPCGatewayAttachment
116         - NatGateway02
117         Type: AWS::EC2::Route
118         Properties:
119           RouteTableId: !Ref PrivateRouteTable02
120           DestinationCidrBlock: 0.0.0.0/0
121           NatGatewayId: !Ref NatGateway02
122
123       NatGateway01:
124         DependsOn:
125         - NatGatewayEIP1
126         - PublicSubnet01
127         - VPCGatewayAttachment
128         Type: AWS::EC2::NatGateway
129         Properties:
130           AllocationId: !GetAtt 'NatGatewayEIP1.AllocationId'
131           SubnetId: !Ref PublicSubnet01
132           Tags:
133           - Key: Name
134             Value: !Sub '${AWS::StackName}-NatGatewayAZ1'
135
136       NatGateway02:
137         DependsOn:
138         - NatGatewayEIP2
139         - PublicSubnet02
140         - VPCGatewayAttachment
141         Type: AWS::EC2::NatGateway
142         Properties:
143           AllocationId: !GetAtt 'NatGatewayEIP2.AllocationId'
144           SubnetId: !Ref PublicSubnet02
145           Tags:
146           - Key: Name
147             Value: !Sub '${AWS::StackName}-NatGatewayAZ2'
148
149       NatGatewayEIP1:
150         DependsOn:
151         - VPCGatewayAttachment
152         Type: 'AWS::EC2::EIP'
153         Properties:
154           Domain: vpc
155
156       NatGatewayEIP2:
157         DependsOn:
158         - VPCGatewayAttachment
159         Type: 'AWS::EC2::EIP'
160         Properties:
161           Domain: vpc
162
163       PublicSubnet01:
164         Type: AWS::EC2::Subnet
165         Metadata:
166           Comment: Subnet 01
167         Properties:
168           MapPublicIpOnLaunch: true
169           AvailabilityZone:
170             Fn::Select:
171             - '0'
172             - Fn::GetAZs:
173                 Ref: AWS::Region
174           CidrBlock:
```

```
            Ref: PublicSubnet01Block
          VpcId:
            Ref: VPC
          Tags:
          - Key: Name
            Value: !Sub "${AWS::StackName}-PublicSubnet01"
          - Key: kubernetes.io/role/elb
            Value: 1

      PublicSubnet02:
        Type: AWS::EC2::Subnet
        Metadata:
          Comment: Subnet 02
        Properties:
          MapPublicIpOnLaunch: true
          AvailabilityZone:
            Fn::Select:
            - '1'
            - Fn::GetAZs:
                Ref: AWS::Region
          CidrBlock:
            Ref: PublicSubnet02Block
          VpcId:
            Ref: VPC
          Tags:
          - Key: Name
            Value: !Sub "${AWS::StackName}-PublicSubnet02"
          - Key: kubernetes.io/role/elb
            Value: 1

      PrivateSubnet01:
        Type: AWS::EC2::Subnet
        Metadata:
          Comment: Subnet 03
        Properties:
          AvailabilityZone:
            Fn::Select:
            - '0'
            - Fn::GetAZs:
                Ref: AWS::Region
          CidrBlock:
            Ref: PrivateSubnet01Block
          VpcId:
            Ref: VPC
          Tags:
          - Key: Name
            Value: !Sub "${AWS::StackName}-PrivateSubnet01"
          - Key: kubernetes.io/role/internal-elb
            Value: 1

      PrivateSubnet02:
        Type: AWS::EC2::Subnet
        Metadata:
          Comment: Private Subnet 02
        Properties:
          AvailabilityZone:
            Fn::Select:
            - '1'
            - Fn::GetAZs:
                Ref: AWS::Region
          CidrBlock:
            Ref: PrivateSubnet02Block
          VpcId:
```

```
            Ref: VPC
          Tags:
          - Key: Name
            Value: !Sub "${AWS::StackName}-PrivateSubnet02"
          - Key: kubernetes.io/role/internal-elb
            Value: 1

      PublicSubnet01RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PublicSubnet01
          RouteTableId: !Ref PublicRouteTable

      PublicSubnet02RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PublicSubnet02
          RouteTableId: !Ref PublicRouteTable

      PrivateSubnet01RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PrivateSubnet01
          RouteTableId: !Ref PrivateRouteTable01

      PrivateSubnet02RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PrivateSubnet02
          RouteTableId: !Ref PrivateRouteTable02

      ControlPlaneSecurityGroup:
        Type: AWS::EC2::SecurityGroup
        Properties:
          GroupDescription: Cluster communication with worker nodes
          VpcId: !Ref VPC

    Outputs:

      SubnetIds:
        Description: Subnets IDs in the VPC
        Value: !Join [ ",", [ !Ref PublicSubnet01, !Ref PublicSubnet02, !Ref
            PrivateSubnet01, !Ref PrivateSubnet02 ] ]

      SecurityGroups:
        Description: Security group for the cluster control plane communication with
            worker nodes
        Value: !Join [ ",", [ !Ref ControlPlaneSecurityGroup ] ]

      VpcId:
        Description: The VPC Id
        Value: !Ref VPC

```

Listing A.1: AWS CloudFormation YAML Template for EKS IPv4

Appendix B: VPC Requirements for EKS Cluster in AWS

The following are the VPC requirements for deploying an EKS Cluster in AWS.

1. The VPC must have sufficient IP addresses for the cluster, any nodes, and Kubernetes resources that are created.
2. If IPv6 is used by Kubernetes, then IPv6 CIDR blocks must be associated with the VPC and corresponding subnets.
3. The VPC must have DNS hostname and DNS resolution support.
4. The subnets must have at least six IP addresses for use by EKS.
5. The subnets must be in at least two AZs.
6. If inbound access is required from internet to pods, then there needs to be at least 1 public subnet with enough available IP addresses to deploy load balancers and ingresses to.

Appendix C: AWS EKS Add-ons

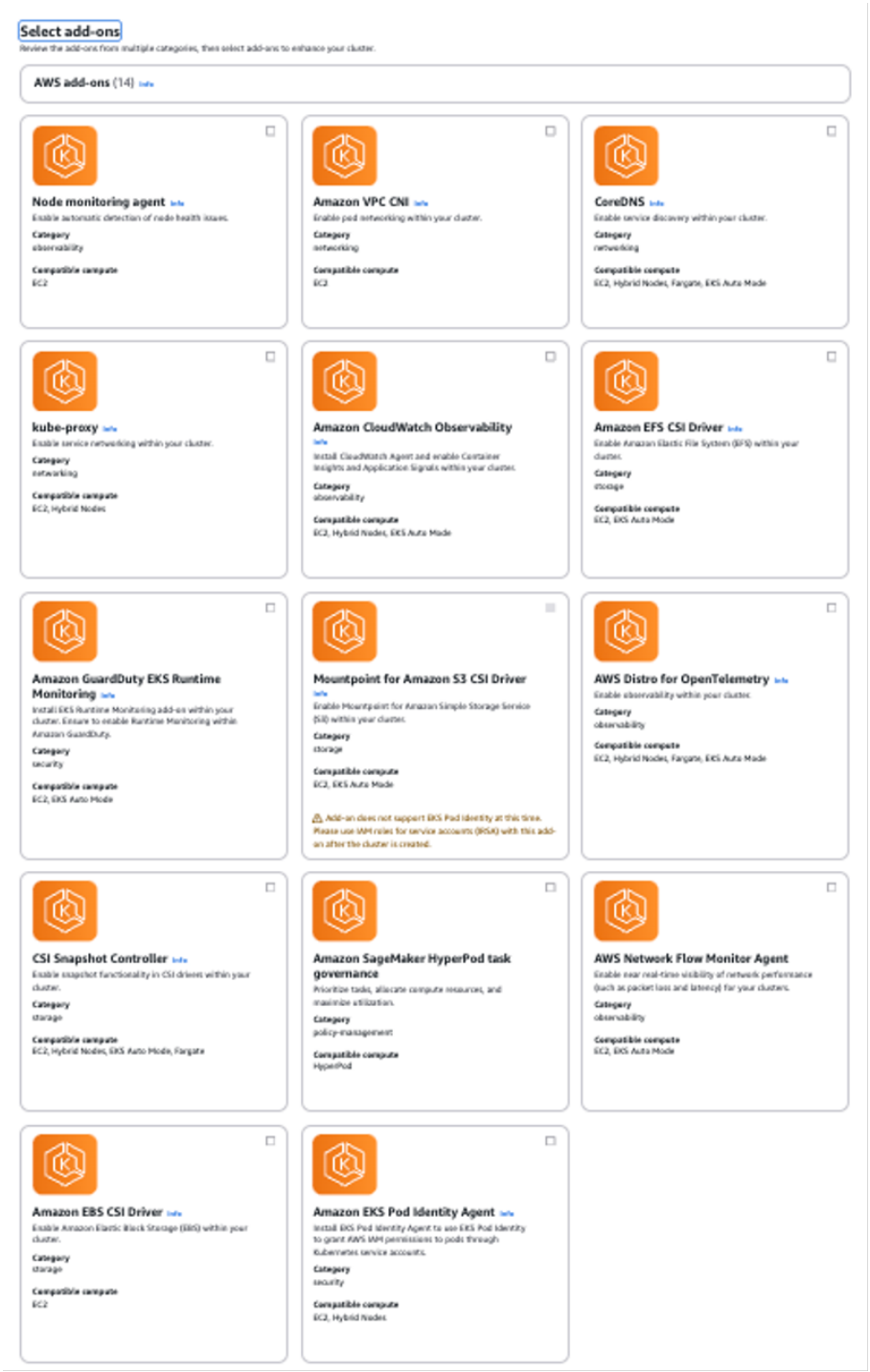

Figure C-1: AWS EKS Optional Add-ons

Appendix C: `cloudkube` README

`cloudkube` Developer Documentation

Initial Setup

The initial setup for `cloudkube` is straightforward and needs to be done only once. Since `cloudkube` runs on Python (`>= 3.11`), it is recommended to use virtual environments to avoid dependency conflicts and keep your global environment clean. Virtual environments can be created using **conda** (recommended), **pipenv**, or **python venv**.

1. **Create and Activate a Virtual Environment:** Use your preferred method (e.g., conda, pipenv, or venv) to create and activate a virtual environment.
2. **Install `cloudkube`:** Run the following command to install `cloudkube` via `pip`.

   ```
   pip install cloudkube
   ```

3. **Verify Installation:** Check if `cloudkube` was installed successfully.

   ```
   pip list | grep cloudkube
   ```

4. **Install AWS CLI:** Follow the official AWS documentation to install the AWS CLI: Getting Started with AWS CLI and configure your cloud credentials.

   ```
   aws configure
   ```

   For additional details, refer to the AWS CLI Welcome Guide.

5. **Install Terraform:**

   - **macOS (Recommended):** Install Terraform using Homebrew:

     ```
     brew tap hashicorp/tap
     brew install hashicorp/tap/terraform
     brew update
     brew upgrade hashicorp/tap/terraform
     ```

   - **Other Platforms:** Terraform can also be installed on other platformrs by following the Terraform Developer Documentation.

6. **Verify Terraform Installation:**

   ```
   terraform -version
   ```

Once all dependencies are installed, you are ready to start using `cloudkube`!

`cloudkube` **Workflow**

1. Initialize `cloudkube`

To start working with cloudkube, run `cloudkube init`. This will bring up an interactive wizard, which will bring you through the provisioning of a cluster.

```
cloudkube init
```

2. Configure Cluster Configuration

a. In the wizard, select **Option 2** to configure the cluster.

b. Provide the following information when prompted:

- Name of the cluster
- Region
- Compute architecture
- Optional add-ons (e.g., persistent volumes, ingress controllers)

c. The configuration details will be saved in a file named `config.json` in the same directory where `cloudkube` is run.

3. Spin Up the Cluster

a. Select **Option 3** in the interactive wizard.

b. `cloudkube` performs the following actions:

- `terraform init`: Installs all required Terraform dependencies and sets up the environment.
- `terraform plan`: Creates a resource plan and seeks user approval before proceeding.
- `terraform apply`: Provisions the required infrastructure, including:
  - An EKS-compliant VPC
  - Relevant permissions and security groups
- Automatically configures the local environment to communicate with the cluster by populating `~/.kube/config`.

4. Verify Cluster Creation

a. Check the cluster creation via the **AWS Console**.

b. Use the following `kubectl` commands for further verification:

- List all pods:

```
kubectl get pods -A
```

- List all nodes:

```
kubectl get nodes
```

5. Apply Kubernetes Manifests

To deploy your application:

a. Ensure your Kubernetes YAML files are stored in the `./kubernetes` directory.

b. Apply the manifests:

```
kubectl apply -f ./kubernetes
```

6. Interact with your Kubernetes Cluster

At this point, we have a fully functional Kubernetes cluster with your deployed application. You should be able to use all kubectl commands to interact with your application. For a list of commands to interact with the cluster, please refer to the Kubernetes Documentation.

7. Teardown and Resource Deprovisioning

Once you are done testing your application:

a. Remove your Kubernetes deployment.

```
kubectl delete -f ./kubernetes
```

b. Run the following command and select **Option 4** in the wizard to deprovision all resources and clean up your local Kubernetes configuration.

```
cloudkube init
```

Notes

- As of version `v1.0`, `cloudkube` only supports **AWS Cloud**. If support for other cloud providers is added in the future, their respective setup steps should be followed.